

# CANDY DIALOGUE ENGINE

## Input UI Customization

1.1.0

Input UI Elements are a special case. Unlike other UI elements, they don't have a specific purpose: they're a UI that accepts and reacts to player input, whatever that input is or what those reactions are supposed to be.

They're basically an open-ended UI element, where you can design their structure, purpose and internal logic however you want.

A default Input scene is provided, which acts as a text input field. However, this is merely an example template: Input UI Elements aren't limited to text inputs -- they can be keypads with buttons, puzzles or even mini-games.

For more information, see the [Player Input](#) guide in the "Commands" folder.

## Nodes

### Root

The root node can be named anything you wish.

However, when the `$Input` command instantiates an input scene, it automatically names the instance the same as the scene file (minus the '.tscn' extension). For example, if you save the scene as "Antikythera Mechanism Puzzle.tscn", the instance will be named "Antikythera Mechanism Puzzle".

This is because the `$Input` command will check if an instance of the provided scene already exists and if so, it will re-use it.

As good practice, we recommend that you always name the root node the same way -- i.e. the same as the scene file, without the '.tscn' suffix. This will prevent confusion if you forget that the `$Input` command automatically renames instances.

### Required Nodes

Due to their open-ended nature, Input UI Elements don't have any required nodes.

# Script

## Base

The default Input UI Element provided with Candy DE has the **Input.gd** script attached to the scene's root node. This script can be used as a template, but can be almost entirely replaced with logic of your own.

To design your own Input UI Elements, you need to understand the variables, signals and functions that the §Input command interacts with.

## Variables

### depth

This stores the nesting depth of the last §Input command that used the input UI.

### caller

This stores a reference to the dialogue instance that last used the input UI through the §Input command.

### default\_z

The default z\_index value. Should be lower than dialogue display elements such as the dialogue box, to ensure dialogue can be displayed over the input UI.

## Signals

### input\_received()

The input\_received() signal is meant to be emitted at strategic points by the input UI, and communicates with the §Input command. To understand how it works, you must first understand the logic of the §Input command:

- 1) The §Input command first instantiates the specified input UI (or 're-activates' it if an instance was already created).
- 2) Then, the §Input command enters a 'while' loop, where it awaits the input\_received() signal. The signal's arguments tell the §Input UI what to do when the signal is detected.

By default, the §Input command has only one pre-programmed behavior: "finished". This causes the §Input command to break out of the 'while' loop. The dialogue engine then moves on to the next dialogue line.

The signal carries three arguments:

- **'depth'** → The nesting depth of the input UI. This ensures that any `$Input` command only reacts to signals emitted by the UI it is handling. You never need to assign this manually or otherwise concern yourself with it, it's all handled automatically.
- **'event'** → The behavior that the signal should trigger in the `$Input` command code. By default, "finished" is the only available option.
- **'parameters'** → An optional, secondary variable for triggering custom behaviors in the `$Input` command. Set to 'null' if unused.

Triggering 'finished' in the `$Input` command makes the dialogue engine move on, but does not close the input UI unless the UI is programmed to do so when the signal is emitted.

This means that you can intentionally design an input UI to remain on screen, and even to continue reacting to player inputs, while dialogue continues in parallel. However, once the dialogue engine continues, the `input_received()` signal will have no effect since the `$Input` command is no longer listening for the signal.

### Several options:

- 1) This is not a problem if the input UI is fully autonomous and does not need to trigger any behaviors inside the `$Input` command.
- 2) You can 'recover' the input UI with another `$Input` command later on during the dialogue: simply add an `$Input` command, and configure it to instantiate the same input UI. Since the input UI hasn't been closed, the command will connect to it instead of creating a new instance.

This second `$Input` command will listen for the input UI's `input_received()` signal, completely normally.

Be mindful that the `$Input` command will call the `setup()` function in the input UI, so ensure your input UI's code is designed to avoid resetting the input UI. As an example, you could provide the second `$Input` command a custom variable as part of its data (e.g. set "Mode" to "Recover") and program the `setup()` command to skip its normal setup logic.

In case you're wondering, nesting depth won't be an issue because the `$Input` command will update the input UI's **depth** to match its own.

- 3) Perhaps simpler: use the `$Await` command to await for the `input_received()` signal. However, if you are concerned that multiple `input_received()` signals may be emitted while the `$Await` command runs, you won't be able to rely on nesting depth to identify the correct signal that the command should react to: unlike `$Input`, `$Await` doesn't update the input UI's **depth**.

## Functions

### setup()

The `setup()` function is called by the `$Input` command, and is therefore required for all Input UI Elements.

In the default `Input.gd` script, this function makes the input UI visible and contains a match statement.

The match statement performs several pre-configurations inside the input UI, depending on the `$Input` command's "Mode" value. By default, it also calls the `config()` function, for further configuration of the UI.

The match statement is helpful if you plan to re-use an input UI for different purposes, or with slightly different settings. Using a match statement isn't necessary for an input UI that only ever has one configuration.

You can modify the entire `setup()` code, or even replace all of it with 'pass' if your input UI requires no additional configuration at runtime.

### config()

The `config()` function is optional. In the default `Input.gd` script, it is called by the `setup()` function and its purpose is to further configure the input UI. Specifically, the permitted text inputs.

You can modify the `config()` function however you want, add cases to its match statement, or omit it entirely from your own custom input UI.

### submit() and cancel()

In the default `Input.gd` script, the purpose of these functions is to determine what happens when the player clicks the "Submit" or "Cancel" buttons featured in the default input UI.

You'll probably want similar functions in your custom input UI, although you might code different logic and these functions might not be called by clicking buttons. For example, you could design your input UI to automatically call such functions when a timer expires, or when the player solves a puzzle.

Notice that both functions emit the `input_received()` signal:

```
caller.input_received.emit(depth, "finish", null)
```

As mentioned previously, this signal communicates with the `$Input` command. The second argument ('event'), with a value of "finish" will cause dialogue to continue, exiting the `$Input` command without deleting the Input UI.

"finish" can be replaced with "close" as a value: this will cause dialogue to continue as well, but will delete the instantiated Input UI.

The signal doesn't cause the input UI to close: this logic must be added to the input UI code (either set 'visible = false' to hide the input UI, or call 'queue\_free()' to delete the input UI instance entirely).

### **check\_input()**

This function is also optional. In the default **Input.gd** script, it checks that any input typed by the player conforms with the allowed input rules.

You may want to pay attention to and replicate the logic for displaying various UI texts, error messages, and checking the blacklist/whitelist for different languages.

### **store\_to\_variable()**

This is another optional function. In the default **Input.gd** script, it stores the submitted input to a variable, for future reference.

This is somewhat important: as previously mentioned, the `input_received()` signal can communicate with the `$Input` command to trigger various behaviors. However, that requires adding code to the `$Input` command for such behaviors.

As an alternative, you can use a function similar to **store\_to\_variable()** to make the input UI directly modify the values of any variables you want.

Either option works, it depends what you find most practical for any given situation.

## Recap

Creating your own input UI isn't complicated. To summarize:

### Mandatory:

- An input UI can be anything you want: a text input menu, a virtual keypad, a puzzle, a mini-game, etc.
- An input UI has no required nodes.
- The script attached to the input UI requires the following variables: **default\_z**, **caller** and **depth**.
- The script attached to an input UI requires a **setup()** function, called by the `$Input` command. The **setup()** function can feature any code you want, or just 'pass' if you don't need the function to do anything.
- The code to hide or close the input UI must be located in and triggered by the input UI itself, when appropriate. The `$Input` command or the dialogue engine don't handle this.
- An input UI must emit the `input_received()` signal to 'end' the `$Input` command and allow dialogue to continue. The signal should always be written like this:

**caller.input\_received.emit(depth, "finish", null)**

(Optionally: replace "finish" with "close" to also delete the instantiated Input UI)

### Optional:

- The `$Input` command passes some of its own data to the input UI: "Variable", "Mode", "Instructions", "Placeholder" and "Text". Your input UI code can use these variables in any manner you want to configure itself.
- The `input_received()` signal can be configured to make the `$Input` command trigger custom behaviors.
- Those custom behaviors must be added by yourself to the `$Input` command's code.
- Instead on relying only on custom behaviors, it can be simpler to program the input UI to directly modify variables, call functions, or perform any other tasks you require.
- You can allow dialogue to continue while the input UI remains open, and while the player interacts with it.

As a reminder, by default, for all data keys (except for "File"), the `$Input` command requires using the 'super' variable symbols to replace the data with a variable value at runtime. This allows you to write and pass variable references verbatim to your code if needed.